Esercizi per la verifica scritta - Sistemi e Reti

Esercizi per la verifica scritta — B1, pag. 101–102

Ollari, *Corso di Sistemi e Reti* vol. 2 — capitolo B1 «Il livello 3: Rete». Per ogni esercizio: **traccia** → **metodo** → **svolgimento passo passo** → **risultato**.

Convenzioni usate in tutto il documento

Campi di una riga di TdI: `Add | P | h | c`

campo	significato
`Add`	indirizzo (o gruppo di indirizzi, es. `B÷D`) di destinazione
`P`	porta di uscita del router
`h`	**hops**: numero di router **intermedi** da attraversare oltre a quello locale. `h = 0` ⟹ destinazione direttamente connessa
`c`	costo: somma dei costi delle linee attraversate. Se non specificato, ogni linea costa 1 (quindi `c = h`)

Intestazioni di un pacchetto (notazione del libro, es. p.89):

```
 MACdest  MACmitt | IPmitt  IPdest | dati
└─── livello 2 ──┘└─ livello 3 ──┘
```

Regola d'oro: **l'intestazione di livello 3 non cambia mai lungo il percorso**; l'intestazione di livello 2 viene riscritta **ad ogni tratta**.

Look-up table CONS: righe `(Pin, Lin) → (Pout, Lout)`. Le label sono **locali alla coppia (porta, direzione)**: la stessa L può essere riusata su porte diverse o in direzione opposta.

PARTE 1 — Instradamento CNLS (es. 1–4)

Esercizio 1 — TdI dei router R1, R2, R3, R4

Traccia. Dato lo schema di rete, determinare le TdI dei quattro router.

Topologia letta dalla figura

Router	Porte
R1	`P1` → host A · `P2` → R2.P4
R2	`P1` → LAN(B, C, D) · `P2` → R4.P3 · `P3` → R3.P1 · `P4` → R1.P2
R3	`P1` → R2.P3 · `P2` → R4.P2 · `P3` → LAN(E, F, G)
R4	`P1` → host H · `P2` → R3.P2 · `P3` → R2.P2

Metodo (4 passi, sempre lo stesso)

1. Elenca i **gruppi di indirizzi** (host raggruppati per LAN): `A`, `B÷D`, `E÷G`, `H`. Un gruppo per LAN ⟹ una riga per gruppo, non una riga per host. 2. Per ogni router e per ogni gruppo, trova il **cammino minimo** sul grafo. 3. Scrivi nella colonna `P` la **prima** porta del cammino (quella locale). 4. `h` = numero di router attraversati **dopo** quello locale; `c = h` (linee a costo 1).

Svolgimento

R1 — è attaccato solo a R2. `A` è locale su P1 → h = 0. Per `B÷D`: R1 → R2 (che le ha in locale) ⟹ un router intermedio, h = 1. Per `E÷G`: R1 → R2 → R3 ⟹ h = 2. Per `H`: R1 → R2 → R4 ⟹ h = 2.

R2 — è il centro stella: raggiunge tutti in un solo salto.

R3 — `E÷G` locale; `H` via R4 (P2, h = 1); `B÷D` via R2 (P1, h = 1); `A` via R2 → R1 (P1, h = 2).

R4 — `H` locale; `E÷G` via R3 (P2, h = 1); `B÷D` via R2 (P3, h = 1); `A` via R2 → R1 (P3, h = 2).

Risultato

TdI R1	P	h	c		TdI R2	P	h	c
A	P1	0	0		A	P4	1	1
B÷D	P2	1	1		B÷D	P1	0	0
E÷G	P2	2	2		E÷G	P3	1	1
H	P2	2	2		H	P2	1	1

> R1 ha **solo due porte** (P1 e P2): tutto ciò che non è A esce per forza da P2. È un caso limite utile per capire che la TdI di un router "foglia" degenera in una sola **default route**.

TdI R3	P	h	c		TdI R4	P	h	c
A	P1	2	2		A	P3	2	2
B÷D	P1	1	1		B÷D	P3	1	1
E÷G	P3	0	0		E÷G	P2	1	1
H	P2	1	1		H	P1	0	0

Rotte alternative (utili per la fault tolerance, da aggiungere come righe in più con `h / c` peggiori): R3 può raggiungere `H` anche via P1 (R2 → R4, h = 2), e R4 può raggiungere `E÷G` anche via P3 (R2 → R3, h = 2).

Esercizio 2 — Intestazioni L2/L3 da A a G

Traccia. Sullo schema dell'es. 1, mostrare per ogni tratta l'intestazione di livello 2 e 3 di un pacchetto da A a G.

Metodo. Ricava il cammino dalle TdI dell'es. 1: R1 instrada `E÷G` su P2, R2 su P3, R3 le ha in locale su P3. Cammino: **A** → **R1** → **R2** → **R3** → **G**. Poi, per ogni tratta, scrivi `MACdest MACmitt │ IPmitt IPdest`: il livello 3 resta `A G` sempre, il livello 2 diventa (indirizzo dell'apparato alla fine della tratta, indirizzo dell'apparato all'inizio della tratta).

Svolgimento

#	Tratta	Intestazione
1	LAN di A → R1 (porta P1)	`r1 a │ A G`
2	R1 → R2 (P2 di R1 → P4 di R2)	`r2 r1 │ A G`
3	R2 → R3 (P3 di R2 → P1 di R3)	`r3 r2 │ A G`
4	R3 → LAN di G (P3 di R3)	`g r3 │ A G`

Da dire alla verifica: cambiano solo gli indirizzi MAC perché ogni tratta è una rete fisica diversa (Ethernet, seriale, …) con il proprio piano di indirizzamento; gli indirizzi IP identificano gli **estremi** della comunicazione e devono sopravvivere fino a destinazione.

Esercizio 3 — TdI dei router (secondo schema)

Topologia letta dalla figura

Router	Porte
R1	`P1` → host A · `P2` → R2.P4
R2	`P1` → host B · `P2` → R4.P1 · `P3` → R3.P1 · `P4` → R1.P2
R3	`P1` → R2.P3 · `P2` → R4.P3 · `P3` → LAN(F, G, H)
R4	`P1` → R2.P2 · `P2` → LAN(C, D, E) · `P3` → R3.P2

Gruppi: `A`, `B`, `C÷E`, `F÷H`. Stesso metodo dell'es. 1.

Risultato

TdI R1	P	h	c		TdI R2	P	h	c
A	P1	0	0		A	P4	1	1
B	P2	1	1		B	P1	0	0
C÷E	P2	2	2		C÷E	P2	1	1
F÷H	P2	2	2		F÷H	P3	1	1

TdI R3	P	h	c		TdI R4	P	h	c
A	P1	2	2		A	P1	2	2
B	P1	1	1		B	P1	1	1
C÷E	P2	1	1		C÷E	P2	0	0
F÷H	P3	0	0		F÷H	P3	1	1

Qui R2, R3, R4 formano un **triangolo**: ogni riga con `h = 1` ha sempre un'alternativa con `h = 2` passando dal terzo router del triangolo.

Esercizio 4 — Intestazioni L2/L3 da E ad A

E sta sulla LAN attaccata a R4.P2. Dalle TdI: R4 instrada `A` su P1 (→ R2), R2 su P4 (→ R1), R1 ha `A` in locale su P1. Cammino: **E** → **R4** → **R2** → **R1** → **A**.

#	Tratta	Intestazione
1	LAN di E → R4 (P2)	`r4 e | E A`
2	R4 → R2 (P1 di R4 → P2 di R2)	`r2 r4 | E A`
3	R2 → R1 (P4 di R2 → P2 di R1)	`r1 r2 | E A`
4	R1 → A (P1 di R1)	`a r1 | E A`

PARTE 2 — Reti CONS e label swapping (es. 5–10)

Metodo generale per le look-up table

Nel **label swapping** l'IS non guarda gli indirizzi: usa la coppia **(porta d'ingresso, label del pacchetto)** come **indice** per leggere la coppia **(porta d'uscita, nuova label)**. Quindi:

1. Traccia il cammino fisico del circuito virtuale sullo schema. 2. Per ogni switch attraversato scrivi una riga `(Pin, Lin) → (Pout, Lout)`. 3. Scegli le label rispettando **un solo vincolo**: su una data porta d'ingresso, due circuiti diversi non possono avere la stessa label (altrimenti l'indice non sarebbe univoco). Label uguali su porte diverse sono legittime. 4. La label cambia ad ogni salto: è questo lo **swapping**.

I circuiti sono **unidirezionali**: `A-B` e `B-A` richiedono righe distinte (vedi es. 6).

Esercizio 5 — Determinare le look-up table

Topologia. `A–S1.P3` · `B–S1.P1` · `S1.P2–S2.P1` · `C–S2.P2` · `S2.P3–S3.P1` · `S3.P3–D` · `S3.P2–E`

Circuiti richiesti: A-C, A-E, B-C, B-D, C-E.

Svolgimento circuito per circuito (label assegnate in ordine di comparsa)

Circuito	Cammino	Righe generate
A-C	S1 → S2	S1 `(3,1)→(2,1)` ; S2 `(1,1)→(2,1)`
A-E	S1 → S2 → S3	S1 `(3,2)→(2,2)` ; S2 `(1,2)→(3,1)` ; S3 `(1,1)→(2,1)`
B-C	S1 → S2	S1 `(1,1)→(2,3)` ; S2 `(1,3)→(2,2)`
B-D	S1 → S2 → S3	S1 `(1,2)→(2,4)` ; S2 `(1,4)→(3,2)` ; S3 `(1,2)→(3,1)`
C-E	S2 → S3	S2 `(2,1)→(3,3)` ; S3 `(1,3)→(2,2)`

Risultato

Switch S1

in P	in L	out P	out L	(circuito)
3	1	2	1	A-C
3	2	2	2	A-E
1	1	2	3	B-C
1	2	2	4	B-D

Switch S2

in P	in L	out P	out L	(circuito)
1	1	2	1	A-C
1	2	3	1	A-E
1	3	2	2	B-C
1	4	3	2	B-D
2	1	3	3	C-E

Switch S3

in P	in L	out P	out L	(circuito)
1	1	2	1	A-E
1	2	3	1	B-D
1	3	2	2	C-E

Sequenze di label — A-C: 1-1 · A-E: 2-1-1 · B-C: 1-3-2 · B-D: 2-4-2-1 · C-E: 1-3-2.

> Le label in ingresso a S2 sulla porta P1 sono tutte diverse (1, 2, 3, 4): è la condizione di univocità. Le label 1 e 2 in uscita su P2 e su P3 di S2 possono ripetersi perché sono su porte diverse.

Esercizio 6 — Determinare le look-up table (secondo schema)

Topologia. `A–S1.P1` · `S1.P2–S2.P1` · `C–S2.P2` · `B–S2.P4` · `S2.P3–S3.P1` · `S3.P3–D` · `S3.P2–E`

Circuiti richiesti: A-D, C-E, D-E, A-B, B-A.

Due casi interessanti: * **D-E** ha sorgente e destinazione sullo **stesso switch** (S3): una sola riga, nessuno swapping su altri nodi. * **A-B** e **B-A** sono due circuiti **distinti**, con righe in direzioni opposte (in una rete CONS reale la connessione è full-duplex e le due direzioni hanno label indipendenti).

Svolgimento

Circuito	Cammino	Righe generate
A-D	S1 → S2 → S3	S1 `(1,1)→(2,1)` ; S2 `(1,1)→(3,1)` ; S3 `(1,1)→(3,1)`
C-E	S2 → S3	S2 `(2,1)→(3,2)` ; S3 `(1,2)→(2,1)`
D-E	solo S3	S3 `(3,1)→(2,2)`
A-B	S1 → S2	S1 `(1,2)→(2,2)` ; S2 `(1,2)→(4,1)`
B-A	S2 → S1	S2 `(4,1)→(1,1)` ; S1 `(2,1)→(1,1)`

Risultato

Switch S1

in P	in L	out P	out L	(circuito)
1	1	2	1	A-D
1	2	2	2	A-B
2	1	1	1	B-A

Switch S2

in P	in L	out P	out L	(circuito)
1	1	3	1	A-D
2	1	3	2	C-E
1	2	4	1	A-B
4	1	1	1	B-A

Switch S3

in P	in L	out P	out L	(circuito)
1	1	3	1	A-D
1	2	2	1	C-E
3	1	2	2	D-E

Esercizio 7 — Quali circuiti virtuali sono descritti

Topologia. `A–S1.P1` · `S1.P2–S3.P1` · `B–S2.P1` · `C–S2.P2` · `S2.P3–S3.P2` · `S3.P4–S4.P1` · `S4.P2–F` · `S3.P3–S5.P1` · `S5.P2–D` · `S5.P3–E`

Look-up table date

S2					**S3**					**S5**			
Pin	**Lin**	**Pout**	**Lout**		**Pin**	**Lin**	**Pout**	**Lout**		**Pin**	**Lin**	**Pout**	**Lout**
1	1	3	2		2	3	3	4		1	2	3	4
2	1	3	3		2	1	3	3		1	4	2	1
1	2	3	1		2	2	3	2		1	3	2	4

Metodo. Parti da ogni riga la cui **porta d'ingresso è collegata a un host** (quelle sono le "teste" dei circuiti). Segui il link fisico dalla porta d'uscita, entra nello switch successivo con la label appena scritta, cerca la riga corrispondente, e ripeti finché non arrivi a una porta con un host attaccato.

Svolgimento

Riga S2 `(1,1)→(3,2)` — P1 di S2 è **B**. Esce da P3 con L=2 → entra in S3 dalla **P2** con L=2. S3 ha `(2,2)→(3,2)`: esce da P3 con L=2 → entra in S5 dalla **P1** con L=2. S5 ha `(1,2)→(3,4)`: esce da P3 → **E**. ⟹ **circuito B → E**, sequenza label `1-2-2-4`.

Riga S2 `(2,1)→(3,3)` — P2 di S2 è **C**. → S3 P2 L=3. S3 `(2,3)→(3,4)` → S5 P1 L=4. S5 `(1,4)→(2,1)` → P2 = **D**. ⟹ **circuito C → D**, sequenza `1-3-4-1`.

Riga S2 `(1,2)→(3,1)` — di nuovo **B**. → S3 P2 L=1. S3 `(2,1)→(3,3)` → S5 P1 L=3. S5 `(1,3)→(2,4)` → P2 = **D**. ⟹ **circuito B → D**, sequenza `2-1-3-4`.

Risultato: sono descritti 3 circuiti virtuali — B-E, C-D, B-D. S1 e S4 hanno look-up table vuote: gli host **A** e **F** non partecipano a nessun circuito.

Esercizio 8 — Aggiornare le look-up table

Nuovi collegamenti: C-E, F-A, D-E, B-F, A-F. Le label già occupate (es. 7) sono: S2 in P1 → {1,2}, in P2 → {1}; S3 in P2 → {1,2,3}, out P3 → {2,3,4}; S5 in P1 → {2,3,4}, out P2 → {1,4}, out P3 → {4}.

Circuito	**Cammino**	**Righe nuove**
C-E	S2 → S3 → S5	S2 `(2,2)→(3,4)` ; S3 `(2,4)→(3,5)` ; S5 `(1,5)→(3,1)`
F-A	S4 → S3 → S1	S4 `(2,1)→(1,1)` ; S3 `(4,1)→(1,1)` ; S1 `(2,1)→(1,1)`

Circuito	Cammino	Righe nuove
D-E	solo S5	S5 `(2,1)→(3,2)`
B-F	S2 → S3 → S4	S2 `(1,3)→(3,5)` ; S3 `(2,5)→(4,1)` ; S4 `(1,1)→(2,1)`
A-F	S1 → S3 → S4	S1 `(1,1)→(2,1)` ; S3 `(1,1)→(4,2)` ; S4 `(1,2)→(2,2)`

Tabelle complete dopo l’aggiornamento

S1 (prima vuota)

in P	in L	out P	out L	
1	1	2	1	A-F
2	1	1	1	F-A

S2

in P	in L	out P	out L	
1	1	3	2	B-E **(preesistente)**
2	1	3	3	C-D **(preesistente)**
1	2	3	1	B-D **(preesistente)**
2	2	3	4	C-E **(nuovo)**
1	3	3	5	B-F **(nuovo)**

S3

in P	in L	out P	out L	
2	3	3	4	C-D **(preesistente)**
2	1	3	3	B-D **(preesistente)**
2	2	3	2	B-E **(preesistente)**
2	4	3	5	C-E **(nuovo)**
4	1	1	1	F-A **(nuovo)**
2	5	4	1	B-F **(nuovo)**
1	1	4	2	A-F **(nuovo)**

S4 (prima vuota)

in P	in L	out P	out L	
2	1	1	1	F-A

in P	in L	out P	out L	
1	1	2	1	B-F
1	2	2	2	A-F

S5

in P	in L	out P	out L	
1	2	3	4	B-E **(preesistente)**
1	4	2	1	C-D **(preesistente)**
1	3	2	4	B-D **(preesistente)**
1	5	3	1	C-E **(nuovo)**
2	1	3	2	D-E **(nuovo)**

Esercizio 9 — Quali circuiti virtuali sono descritti

Topologia. `A–S1.P1` · `S1.P2–S3.P1` · `B–S2.P1` · `S2.P2–S3.P2` · `C–S4.P1` · `E–S4.P3` · `S4.P2–S3.P5` · `S3.P3–D` · `S3.P4–S5.P1` · `S5.P2–F`

Look-up table date

S1					**S3**					**S4**			
Pin	**Lin**	**Pout**	**Lout**		**Pin**	**Lin**	**Pout**	**Lout**		**Pin**	**Lin**	**Pout**	**Lout**
1	1	2	1		1	1	5	1		2	1	1	1
2	2	1	2		5	2	1	2		3	2	2	2
2	3	1	3		3	3	1	3					

Svolgimento

S1 `(1,1)→(2,1)` — P1 di S1 è **A**. → S3 P1 L=1. S3 `(1,1)→(5,1)` → S4 P2 L=1. S4 `(2,1)→(1,1)` → P1 = **C**. ⟹ **A** → **C**, label `1-1-1`.

S4 `(3,2)→(2,2)` — P3 di S4 è **E**. → S3 P5 L=2. S3 `(5,2)→(1,2)` → S1 P2 L=2. S1 `(2,2)→(1,2)` → P1 = **A**. ⟹ **E** → **A**, label `2-2-2`.

S3 `(3,3)→(1,3)` — P3 di S3 è **D**. → S1 P2 L=3. S1 `(2,3)→(1,3)` → **A**. ⟹ **D** → **A**, label `3-3`.

Risultato: 3 circuiti — A-C, E-A, D-A. S2 e S5 sono vuoti: **B** e **F** non partecipano a nessun circuito.

Esercizio 10 — Aggiornare le look-up table

Nuovi collegamenti: A-D, E-D, A-F, B-D, C-E.

Circuito	Cammino	Righe nuove
A-D	S1 → S3	S1 `(1,2)→(2,2)` ; S3 `(1,2)→(3,1)`
E-D	S4 → S3	S4 `(3,3)→(2,3)` ; S3 `(5,3)→(3,2)`
A-F	S1 → S3 → S5	S1 `(1,3)→(2,3)` ; S3 `(1,3)→(4,1)` ; S5 `(1,1)→(2,1)`
B-D	S2 → S3	S2 `(1,1)→(2,1)` ; S3 `(2,1)→(3,3)`
C-E	solo S4	S4 `(1,1)→(3,1)`

Tabelle complete dopo l’aggiornamento

S1

in P	in L	out P	out L	
1	1	2	1	A-C
2	2	1	2	E-A
2	3	1	3	D-A
1	2	2	2	A-D **(nuovo)**
1	3	2	3	A-F **(nuovo)**

S2 (prima vuota)

in P	in L	out P	out L	
1	1	2	1	B-D

S3

in P	in L	out P	out L	
1	1	5	1	A-C
5	2	1	2	E-A
3	3	1	3	D-A
1	2	3	1	A-D **(nuovo)**
5	3	3	2	E-D **(nuovo)**
1	3	4	1	A-F **(nuovo)**
2	1	3	3	B-D **(nuovo)**

S4

in P	in L	out P	out L	
2	1	1	1	A-C
3	2	2	2	E-A
3	3	2	3	E-D **(nuovo)**
1	1	3	1	C-E **(nuovo)**

S5 (prima vuota)

in P	in L	out P	out L	
1	1	2	1	A-F

PARTE 3 — Backward learning (es. 11–12)

Metodo

Regola unica (routing isolato, algoritmo **backward learning**):

> se da una porta **P** giunge un pacchetto **proveniente** dall'host **X**, allora attraverso **P** si può raggiungere **X** ⟹ aggiungi in TdI la regola `(Add = X, P = P)`.

Il router impara **dal mittente**, non dal destinatario. Se la destinazione non è ancora in TdI, il pacchetto viene inoltrato in **flooding** (su tutte le porte tranne quella d'ingresso).

> **Assunzione usata qui** (standard nella risoluzione a mano): si suppone che ogni pacchetto raggiunga il destinatario lungo il **cammino minimo**, e che ogni router attraversato impari il mittente sulla porta d'ingresso. Con il flooding puro si otterrebbero anche righe spurie e duplicate — vale la pena citarlo alla verifica, ma le TdI "pulite" sono quelle qui sotto.

Esercizio 11 — Backward learning sullo schema dell'es. 1

Traffico: A→B, A→H, B→A, H→D, G→A, G→D, A→G. TdI inizialmente vuote.

#	Pacchetto	Cammino	Regole apprese
1	A→B	A → R1(P1) → R2(P4) → B	R1: `A|P1` · R2: `A|P4`

#	Pacchetto	Cammino	Regole apprese
2	A→H	A → R1(P1) → R2(P4) → R4(P3) → H	R4: `A|P3`
3	B→A	B → R2(P1) → R1(P2) → A	R2: `B|P1` · R1: `B|P2`
4	H→D	H → R4(P1) → R2(P2) → D	R4: `H|P1` · R2: `H|P2`
5	G→A	G → R3(P3) → R2(P3) → R1(P2) → A	R3: `G|P3` · R2: `G|P3` · R1: `G|P2`
6	G→D	G → R3(P3) → R2(P3) → D	**(nessuna nuova)**
7	A→G	A → R1(P1) → R2(P4) → R3(P1) → G	R3: `A|P1`

Risultato

TdI R1	**P**		**TdI R2**	**P**		**TdI R3**	**P**		**TdI R4**	**P**
A	P1		A	P4		G	P3		A	P3
B	P2		B	P1		A	P1		H	P1
G	P2		H	P2						
			G	P3						

Osservazioni da riportare: nessun router impara **D**, **C**, **E**, **F** perché non hanno mai trasmesso. È il limite del backward learning: conosce solo chi ha già parlato, e infatti va sempre accoppiato con il flooding e con un meccanismo di **aging** delle righe.

Esercizio 12 — Backward learning sullo schema dell'es. 3

Stesso traffico: A→B, A→H, B→A, H→D, G→A, G→D, A→G. Qui H e G stanno sulla LAN di R3.P3, D sulla LAN di R4.P2, B su R2.P1.

#	Pacchetto	Cammino	Regole apprese
1	A→B	A → R1(P1) → R2(P4) → B	R1: `A|P1` · R2: `A|P4`

#	Pacchetto	Cammino	Regole apprese
2	A→H	A → R1(P1) → R2(P4) → R3(P1) → H	R3: `A|P1`
3	B→A	B → R2(P1) → R1(P2) → A	R2: `B|P1` · R1: `B|P2`
4	H→D	H → R3(P3) → R4(P3) → D	R3: `H|P3` · R4: `H|P3`
5	G→A	G → R3(P3) → R2(P3) → R1(P2) → A	R3: `G|P3` · R2: `G|P3` · R1: `G|P2`
6	G→D	G → R3(P3) → R4(P3) → D	R4: `G|P3`
7	A→G	A → R1(P1) → R2(P4) → R3(P1) → G	**(nessuna nuova)**

Risultato

TdI R1	**P**		**TdI R2**	**P**		**TdI R3**	**P**		**TdI R4**	**P**
A	P1		A	P4		A	P1		H	P3
B	P2		B	P1		H	P3		G	P3
G	P2		G	P3		G	P3			

PARTE 4 — Distance vector: costi e aggiornamenti (es. 13–16)

Metodo per ricavare i costi mancanti

Ogni riga `Add|P|h|c` di una TdI dice: «per arrivare a `Add` esco da `P`, attraverso `h` router e spendo `c`». Quindi:

- `h = 0` ⟹ `Add` è su una LAN **direttamente connessa** a quella porta;
- `h = 1` ⟹ `c` è **esattamente il costo della linea** verso il router vicino;
- `h ≥ 2` ⟹ `c` è la **somma** dei costi delle linee del cammino.

Si mettono a sistema le equazioni ricavate dalle varie righe.

Esercizio 13 — Costi mancanti delle linee

Topologia. `R1.P2–R2.P2` · `R1.P3–R3.P1` **(costo 7, dato)** · `R2.P1–R4.P2` · `R3.P3–R4.P1`. Porte stub: `R1.P1`, `R3.P2`, `R4.P3`.

Dati.

TdI R1	P	h	c		TdI R2	P	h	c		TdI R4	P	h
A	3	1	7		A	1	2	7		B	1	2
B	1	0	0		B	2	1	6		C	2	2
C	1	0	0									

Svolgimento passo passo

1. **Dove stanno gli host.** R1 ha `B` e `C` con `h = 0` su P1 ⟹ **B e C sono sulla LAN attaccata a R1.P1**. R1 ha `A` con `h = 1` su P3 (→ R3) ⟹ **A è sulla LAN attaccata a R3** (porta stub P2). 2. **Riga R1→A** (`h = 1`, `c = 7`): il costo è quello della sola linea R1–R3 ⟹ conferma il valore **7** già stampato sullo schema. ✔ 3. **Riga R2→B** (`P2`, `h = 1`, `c = 6`): P2 di R2 va a R1, e R1 ha B in locale. Un solo salto ⟹ **costo(R1–R2) = 6**. 4. **Riga R4→B** (`P1`, `h = 2`, `c = 10`): P1 di R4 va a R3, poi R3 → R1 → B. `c = costo(R4–R3) + costo(R3–R1) = costo(R4–R3) + 7 = 10` ⟹ **costo(R3–R4) = 3**. 5. **Riga R4→C** (`P2`, `h = 2`, `c = 10`): P2 di R4 va a R2, poi R2 → R1 → C. `c = costo(R4–R2) + costo(R2–R1) = costo(R4–R2) + 6 = 10` ⟹ **costo(R2–R4) = 4**. 6. **Verifica incrociata** sulla riga R2→A (`P1`, `h = 2`, `c = 7`): R2 → R4 → R3 → A vale `4 + 3 = 7`. ✔ Coerente.

Risultato

Linea	Costo
R1 – R2	**6**
R1 – R3	7 **(dato)**
R2 – R4	**4**
R3 – R4	**3**

> Nota: da R4 sia `B` sia `C` costano 10 per entrambe le strade (R3–R1 = 3+7 = 10 e R2–R1 = 4+6 = 10): è un pareggio, il che rende le due righe della TdI di R4 coerenti comunque si scelga la porta.

Esercizio 14 — TdI aggiornate in base al traffico

Le terne `(Mitt, Dest, h, c)` sono i valori **già accumulati** dal pacchetto quando arriva. Regola: il router aggiunge `Mitt | porta d'ingresso | h | c`, poi inoltra verso `Dest` incrementando `h` di 1 e `c` del costo della linea.

a) su P2 di R3 giunge (X, A, 0, 0)

P2 di R3 è la porta stub su cui sta anche **A** (vedi es. 13, punto 1). ⟹ R3 impara `X | P2 | 0 | 0`; il pacchetto **non viene instradato** (mittente e destinatario sono sulla stessa LAN, la consegna è diretta).

b) su P3 di R4 giunge (Y, B, 1, 9)

1. R4 impara `Y | P3 | 1 | 9`. La sua TdI dice `B → P1` ⟹ esce verso R3 (linea di costo 3). 2. R3 riceve su P3 con `h = 2`, `c = 9 + 3 = 12` ⟹ impara `Y | P3 | 2 | 12`. B sta su R1, R3 esce da P1 (linea di costo 7). 3. R1 riceve su P3 con `h = 3`, `c = 12 + 7 = 19` ⟹ impara `Y | P3 | 3 | 19`. Consegna B sulla LAN di P1.

c) su P1 di R1 giunge (Z, X, 2, 3)

1. R1 impara `Z | P1 | 2 | 3`. **X non è in TdI di R1** ⟹ flooding su P2 e P3. 2. Ramo P3 → R3 riceve con `h = 3`, `c = 3 + 7 = 10` ⟹ impara `Z | P1 | 3 | 10`. R3 conosce X (punto a) ⟹ **consegna riuscita**. ✔ 3. Ramo P2 → R2 riceve con `h = 3`, `c = 3 + 6 = 9` ⟹ impara `Z | P2 | 3 | 9`. Non conosce X ⟹ rilancia su P1 (→ R4). 4. R4 riceve su P2 con `h = 4`, `c = 9 + 4 = 13` ⟹ impara `Z | P2 | 4 | 13`. Rilancia su P1 (→ R3). 5. R3 riceve il **duplicato** su P3 con `h = 5`, `c = 16`: peggiore della riga già presente (`h = 3`, `c = 10`) ⟹ **scartato**.

Risultato finale

TdI R1	**P**	**h**	**c**		**TdI R2**	**P**	**h**	**c**
A	3	1	7		A	1	2	7
B	1	0	0		B	2	1	6
C	1	0	0		**Z**	**2**	**3**	**9**
Y	**3**	**3**	**19**					
Z	**1**	**2**	**3**					

TdI R3	**P**	**h**	**c**		**TdI R4**	**P**	**h**	**c**
X	**2**	**0**	**0**		B	1	2	10
Y	**3**	**2**	**12**		C	2	2	10
Z	**1**	**3**	**10**		**Y**	**3**	**1**	**9**
					Z	**2**	**4**	**13**

Esercizio 15 — Costi mancanti delle linee (secondo schema)

Topologia. `R1.P3–R2.P1` · `R1.P4–R3.P2` · `R2.P3–R3.P3` **(costo 3, dato)**. Porte stub: `R1.P1`, `R1.P2`, `R2.P2`, `R3.P1`.

Dati.

TdI R1	P	h	c		TdI R2	P	h	c		TdI R3	P	h
B	4	1	6		A	3	1	3		A	1	0
C	2	0	0		B	1	2	10		B	1	0

Svolgimento

1. **Dove stanno gli host.** R3 ha `A` e `B` con `h = 0` su P1 ⟹ **A e B sono sulla LAN attaccata a R3.P1**. R1 ha `C` con `h = 0` su P2 ⟹ **C è sulla LAN attaccata a R1.P2**. 2. **Riga R2→A** (`P3`, `h = 1`, `c = 3`): P3 di R2 va a R3 che ha A in locale. Un solo salto ⟹ conferma **costo(R2–R3) = 3**. ✔ 3. **Riga R1→B** (`P4`, `h = 1`, `c = 6`): P4 di R1 va a R3 che ha B in locale. Un solo salto ⟹ **costo(R1–R3) = 6**. 4. **Riga R2→B** (`P1`, `h = 2`, `c = 10`): P1 di R2 va a R1, poi R1 → R3 → B. `c = costo(R2–R1) + costo(R1–R3) = costo(R2–R1) + 6 = 10` ⟹ **costo(R1–R2) = 4**.

Risultato

Linea	Costo
R1 – R2	**4**
R1 – R3	**6**
R2 – R3	3 **(dato)**

> Attenzione (ottima osservazione da fare alla verifica): la riga `B|P1|2|10` di R2 **non è il cammino ottimo** — via P3 (R3 diretto) B costerebbe 3 con `h = 1`. È una **rotta alternativa**, tenuta in TdI per fault tolerance. Proprio perché è una rotta più lunga permette di ricavare il costo R1–R2.

Esercizio 16 — TdI aggiornate in base al traffico

a) su P2 di R2 giunge (X, A, 0, 0)

1. P2 di R2 è una porta stub ⟹ R2 impara `X | P2 | 0 | 0`. 2. A sta su R3: R2 esce da P3 (costo 3). 3. R3 riceve su P3 con `h = 1`, `c = 3` ⟹ impara `X | P3 | 1 | 3`; consegna A sulla LAN di P1. ✔

b) su P2 di R1 giunge (Y, B, 1, 9)

1. P2 di R1 è la stub di C ⟹ R1 impara `Y | P2 | 1 | 9`. 2. B sta su R3: R1 esce da P4 (costo 6). 3. R3 riceve su P2 con `h = 2`, `c = 9 + 6 = 15` ⟹ impara `Y | P2 | 2 | 15`; consegna B sulla LAN di P1. ✔

c) su P1 di R3 giunge (Z, C, 2, 3)

1. P1 di R3 è la LAN di A e B ⟹ R3 impara `Z | P1 | 2 | 3`. 2. **C non è in TdI di R3** ⟹ flooding su P2 (→R1) e P3 (→R2). 3. Ramo P2 → R1 riceve su P4 con `h = 3`, `c = 3 + 6 = 9` ⟹ impara `Z | P4 | 3 | 9`. R1 conosce C su P2 ⟹ **consegna riuscita**. ✔ 4. Ramo P3 → R2 riceve su P3 con `h = 3`,

`c = 3 + 3 = 6` ⟹ impara `Z | P3 | 3 | 6`. Non conosce C ⟹ rilancia su P1 (→R1). 5. R1 riceve il **duplicato** su P3 con `h = 4`, `c = 6 + 4 = 10`: peggiore della riga già presente (`h = 3`, `c = 9`) ⟹ **scartato**.

Risultato finale

TdI R1	P	h	c		TdI R2	P	h	c		TdI R3	P	h
B	4	1	6		A	3	1	3		A	1	0
C	2	0	0		B	1	2	10		B	1	0
Y	**2**	**1**	**9**		**X**	**2**	**0**	**0**		**X**	**3**	**1**
Z	**4**	**3**	**9**		**Z**	**3**	**3**	**6**		**Y**	**2**	**2**
									Z	**1**	**2**	**3**

PARTE 5 — Link State (es. 17)

Esercizio 17 — LSP, TdI e LSPdbase

Traccia. Determinare il LSP di ogni router, le TdI di ogni router e il LSPdbase della rete. Le linee senza costo indicato hanno costo 0.

Topologia con costi

Linea	Porte	Costo
R1 – R2	R1.P2 ↔ R2.P4	3
R1 – R3	R1.P3 ↔ R3.P4	5
R2 – R3	R2.P3 ↔ R3.P1	3
R2 – R4	R2.P2 ↔ R4.P3	4
R3 – R4	R3.P2 ↔ R4.P2	2
R1 – A (LAN)	R1.P1	0
R2 – B÷D (LAN)	R2.P1	0
R3 – E÷G (LAN)	R3.P3	0
R4 – H (LAN)	R4.P1	0

Passo 1 — LSP (Link State Packet) di ogni router

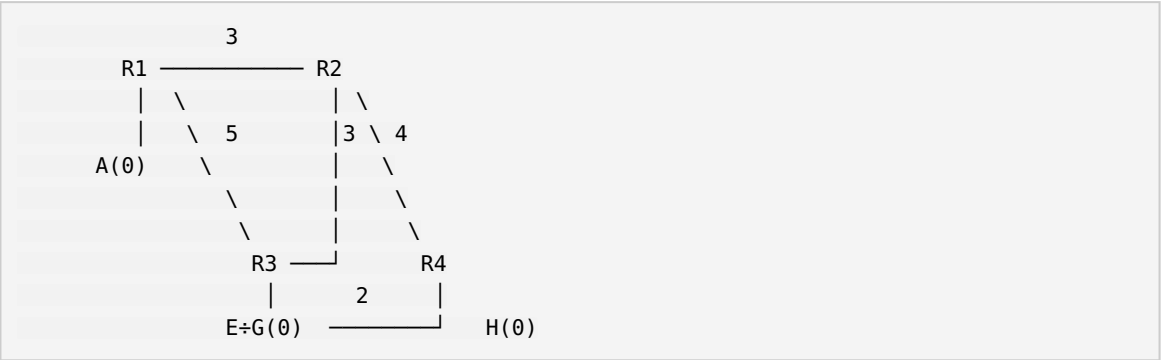
Un LSP contiene **solo** ciò che il router vede in prima persona: l'elenco dei propri vicini con il costo della linea che li collega. È generato da ogni router e diffuso in **flooding** a tutta la rete.

LSP di R1		**LSP di R2**		**LSP di R3**		**LSP di R4**	
vicino	costo (porta)	vicino	costo (porta)	vicino	costo (porta)	vicino	costo (porta)
A	0 (P1)	B÷D	0 (P1)	R2	3 (P1)	H	0 (P1)
R2	3 (P2)	R4	4 (P2)	R4	2 (P2)	R3	2 (P2)
R3	5 (P3)	R3	3 (P3)	E÷G	0 (P3)	R2	4 (P3)
		R1	3 (P4)	R1	5 (P4)		

Passo 2 — LSPdbase

Il **LSPdbase** è semplicemente l'insieme dei quattro LSP sopra. Dopo il flooding **ogni router ne possiede una copia identica**: è questa la differenza sostanziale con il distance vector, dove ogni router conosce solo le distanze riferite dai vicini e non la topologia.

Rappresentato come grafo pesato:



Archi: `R1–R2 = 3`, `R1–R3 = 5`, `R2–R3 = 3`, `R2–R4 = 4`, `R3–R4 = 2`.

Passo 3 — TdI: algoritmo di Dijkstra da ogni router

Metodo. Da ogni router si applica Dijkstra sul grafo del LSPdbase; per ogni destinazione si annota la **prima porta** del cammino minimo, il numero di router intermedi `h` e il costo totale `c`.

Da R1

destinazione	**cammini possibili**	**migliore**
A	locale	P1, h = 0, c = 0
B÷D (su R2)	R1→R2 = 3	P2, h = 1, **c = 3**
E÷G (su R3)	R1→R3 = **5** · R1→R2→R3 = 3+3 = 6	P3, h = 1, **c = 5**
H (su R4)	R1→R2→R4 = 3+4 = **7** · R1→R3→R4 = 5+2 = **7**	pareggio: P2 **o** P3, h = 2, c = 7

Tdl R1	P	h	c
A	P1	0	0
B÷D	P2	1	3
E÷G	P3	1	5
H	P2 (alt. P3)	2	7

Da R2

destinazione	cammini	migliore
A (su R1)	R2→R1 = 3	P4, h = 1, c = 3
B÷D	locale	P1, h = 0, c = 0
E÷G (su R3)	R2→R3 = **3** · R2→R4→R3 = 4+2 = 6	P3, h = 1, c = 3
H (su R4)	R2→R4 = **4** · R2→R3→R4 = 3+2 = 5	P2, h = 1, c = 4

Tdl R2	P	h	c
A	P4	1	3
B÷D	P1	0	0
E÷G	P3	1	3
H	P2	1	4

Da R3

destinazione	cammini	migliore
A (su R1)	R3→R1 = **5** · R3→R2→R1 = 3+3 = 6	P4, h = 1, c = 5
B÷D (su R2)	R3→R2 = **3** · R3→R4→R2 = 2+4 = 6	P1, h = 1, c = 3
E÷G	locale	P3, h = 0, c = 0
H (su R4)	R3→R4 = **2** · R3→R2→R4 = 3+4 = 7	P2, h = 1, c = 2

Tdl R3	P	h	c
A	P4	1	5
B÷D	P1	1	3
E÷G	P3	0	0
H	P2	1	2

Da R4

destinazione	cammini	migliore
A (su R1)	R4→R2→R1 = 4+3 = **7** · R4→R3→R1 = 2+5 = **7**	pareggio: P3 **o** P2, h = 2, c = 7
B÷D (su R2)	R4→R2 = **4** · R4→R3→R2 = 2+3 = 5	P3, h = 1, c = 4
E÷G (su R3)	R4→R3 = **2**	P2, h = 1, c = 2
H	locale	P1, h = 0, c = 0

TdI R4	P	h	c
A	P3 (alt. P2)	2	7
B÷D	P3	1	4
E÷G	P2	1	2
H	P1	0	0

Osservazioni finali da riportare

- La linea **R1–R3 (costo 5)** non viene mai usata da R2 e R4: la coppia R1–R2–R3 costa 6 contro 5, quindi resta utile solo a R1 stesso.
- I due **pareggi a 7** (R1↔R4) sono un tipico caso di **equal-cost multipath**: entrambe le righe possono coesistere in TdI e realizzare load balancing.
- Vantaggio del link state rispetto al distance vector: ogni router calcola **da sé** l'albero dei cammini minimi sulla topologia completa ⟹ convergenza rapida e nessun problema di **count to infinity**.